SMART FARMER ASSISTANT

Module-wise Workflow, Core Logic and Technology Explanation

Purpose of this document

This guide explains every major project module in simple presentation language. It identifies the main files, important code lines, workflows, technologies used, and ready-to-speak answers for faculty questions.

Main framework: Django | **Database:** SQLite | **AI:** Machine Learning + Deep Learning

Contents

One-line project introduction
Smart Farmer Assistant is a Django-based web application that helps farmers manage crops, analyse farm data, check weather and mandi prices, predict diseases, receive fertilizer guidance, read farming news and scan leaf images using AI.

1. Overall Project Architecture

The project follows Django's normal request-response flow.

User action ↓ URL ↓ Django view ↓ Model / API / ML logic ↓ Database or prediction ↓ HTML result

Simple explanation: When the user clicks a button or submits a form, Django checks the URL, calls the correct view function, performs the required logic, and sends the final data to an HTML template.

- **core/** - dashboard, weather, market prices, fertilizer guidance, news, profile and crop library.
- **crops/** - crop CRUD, analytics, machine-learning disease prediction and PDF report.
- **leaf_scanner/** - deep-learning leaf image classification.
- **templates/** - frontend pages displayed to the user.
- **SmartFarmer/** - project settings, main URL routing and deployment configuration.

```
SmartFarmer/urls.py
↓
core/urls.py, crops/urls.py, leaf_scanner/urls.py
↓
view function
↓
model / API / ML engine
↓
template
```

The application uses **Django ORM**, so most database work is performed through Python objects instead of raw SQL.

2. Dashboard Module

Purpose: The dashboard gives the logged-in farmer a quick summary of the entire farm.

- Total cultivated land
- Total investment
- Expected revenue
- Recent crops
- Crop growth progress
- Recent disease predictions
- Profile-based farm information

Main file: `core/views.py`

Main function: `dashboard(request)`

Farmer opens dashboard ↓ Django reads farmer's crops ↓ Totals are calculated ↓ Recent records are selected ↓ dashboard.html displays summary

```
user_crops = Crop.objects.filter(farmer=request.user)

total_land = sum(crop.land_size for crop in user_crops)
total_investment = sum(crop.investment_cost for crop in user_crops)
expected_revenue = sum(crop.estimated_revenue for crop in user_crops)
```

The first line retrieves only the current farmer's crop records. The next lines calculate totals for the dashboard.

"The dashboard collects the logged-in farmer's crop and prediction records from SQLite, calculates important totals and displays a complete farm summary."

3. My Crops Module (CRUD)

Purpose: This module allows a farmer to manage personal crop records.

CRUD means: Create, Read, Update and Delete.

- Add a crop
- View crop details
- Update crop information
- Delete a crop
- Search by crop name or variety
- Filter by crop status
- See growth progress

Main files: `crops/models.py, crops/forms.py, crops/views.py, crops/urls.py`

Farmer fills form ↓ Django validates input ↓ Crop is linked to logged-in user ↓ Record is saved in SQLite ↓ Crop appears in My Crops

```
crop = form.save(commit=False)
crop.farmer = request.user
crop.save()
```

Security check:

```
crop = get_object_or_404(Crop, pk=pk, farmer=request.user)
```

Search logic:

```
crops = crops.filter(
Q(name__icontains=query) |
Q(variety__icontains=query)
)
```

"My Crops is a secure CRUD module made using Django models, forms and views. Each farmer can manage only their own crop records."

4. Crop Analysis Module

Purpose: Crop Analysis converts stored crop records into useful statistics and interactive charts.

- Total and average land
- Total investment
- Expected yield
- Expected revenue
- Highest-revenue crop
- Highest-yield crop
- Crop status distribution
- Sowing timeline

Technologies: Pandas, Plotly and Django ORM

Crop records from SQLite ↓ Convert to Pandas DataFrame ↓ Calculate totals and averages ↓ Plotly creates charts ↓ Charts become HTML ↓ Analytics page displays them

```
df = pd.DataFrame(data_list)

df['land_size'].sum()
df['investment'].mean()
df['revenue'].max()
```

```
fig_land = px.pie(...)
fig_inv = go.Figure(data=[go.Bar(...)])
fig_rev = go.Figure(data=[go.Bar(...)])
fig_status = px.pie(...)
fig_timeline = px.line(...)
```

“I used Pandas to analyse crop records and Plotly to create interactive pie charts, bar charts and line charts.”

5. Crop Library Module

Purpose: The Crop Library provides reference information about different crops.

- Crop name and Gujarati name
- Season
- Suitable soil
- Temperature range
- Rainfall requirement
- Sowing and harvesting time
- Expected yield
- Common diseases
- Suitable fertilizers

Reference data stored in SQLite ↓ Django loads crops ↓ User searches or selects season ↓ Django ORM filters records ↓ Matching information is displayed

```
crops = crops.filter(
Q(name__icontains=query) |
Q(gujarati_name__icontains=query) |
Q(soil_type__icontains=query) |
Q(common_diseases__icontains=query)
)

crops = crops.filter(season=selected_season)
```

Important clarification
Crop Library is not machine learning. It is a database-based information, search and filtering module.

"The Crop Library stores crop knowledge in SQLite and provides keyword search and season-based filtering through Django ORM."

6. Weather Updates Module

Purpose: This module shows current weather and forecast information useful for farming.

- Temperature
- Humidity
- Rainfall
- Wind speed
- Weather condition
- Forecast
- Agricultural advice
- Search history

Technologies: OpenWeatherMap REST API, Python Requests, JSON, Django and SQLite

Farmer enters location ↓ Django sends API request ↓ OpenWeatherMap returns JSON ↓ Python extracts values ↓ Weather is displayed ↓ Search history is saved

```
query = f"{profile.village}, {profile.district}"

geo_response = requests.get(geo_url, params=geo_params, timeout=8)
weather_response = requests.get(weather_url, params=weather_params, timeout=8)
forecast_response = requests.get(forecast_url, params=forecast_params, timeout=8)

WeatherHistory.objects.create(...)
```

“I integrated the OpenWeatherMap REST API. The farmer's location is sent to the API, weather data is received in JSON format, processed in Python and displayed on the website.”

7. Market Price Module

Purpose: This module provides agricultural mandi price information.

- Crop name
- APMC mandi
- Modal price
- Minimum and maximum price
- Previous price
- Price increase or decrease
- Mandi comparison graph
- Historical price data

API: Government of India Open Government Data platform - api.data.gov.in

Call Government API ↓ Receive JSON records ↓ Keep relevant Gujarat data ↓ Clean crop names ↓ Store in SQLite ↓ Prepare with Pandas ↓ Plotly creates line graph

```
base_url = f'https://api.data.gov.in/resource/{resource_id}'

params = {
'api-key': api_key,
'format': 'json',
'limit': '5000',
'filters[state]': 'Gujarat'
}

response_data = json.loads(response.read().decode('utf-8'))
records = response_data.get('records', [])
```

```
MandiPrice.objects.create(...)
df = pd.DataFrame(data_list)

fig.add_trace(go.Scatter(
x=mandi_list,
y=prices_list,
mode='lines+markers'
))
```

"I integrated the Government of India market-price API to fetch Gujarat APMC records. The JSON data is processed, stored in SQLite and visualized using Plotly."

8. Disease Prediction Module - Machine Learning

Purpose: This module predicts a possible crop disease using environmental and crop-related inputs.

Inputs: Crop name, temperature, humidity, rainfall, soil type and selected algorithm.

- Decision Tree
- Random Forest
- K-Nearest Neighbours (KNN)

Farmer enters conditions ↓ Text values are encoded as numbers ↓ Values go to selected model ↓ Model predicts disease ↓ Confidence is calculated ↓ Advice is returned ↓ Result is saved

```
self.dt_model.fit(self.X, self.y)
self.rf_model.fit(self.X, self.y)
self.knn_model.fit(self.X, self.y)

pred_label = model.predict(features)[0]
proba = model.predict_proba(features)[0]
```

Important project detail
The small training dataset is defined directly inside crops/ml_engine.py. The models are trained in memory when the engine is initialized; this module does not load a separate CSV dataset.

"I used supervised classification. Based on crop, soil and weather conditions, the selected Decision Tree, Random Forest or KNN model predicts a possible disease."

9. Fertilizer Guidance Module

Purpose: This module recommends fertilizer according to crop, soil type and crop growth stage.

- Fertilizer name
- NPK ratio
- Quantity per acre
- Application method
- Best application time
- Safety precautions

Farmer selects crop, soil and stage ↓ System finds matching rule ↓ Quantity is adjusted for soil ↓ Recommendation is saved ↓ Result and history are displayed

Important clarification
This module is rule-based, not machine learning. It uses predefined recommendations and soil multipliers.

```
recommendations_db = {
...
}

# Sandy soil
multiplier = 1.2

# Red soil
multiplier = 1.1

# Alluvial soil
multiplier = 0.95

FertilizerRecommendation.objects.create(...)
```

“The Fertilizer Guidance module uses rule-based logic. It matches crop and growth stage, then adjusts fertilizer quantity according to soil type.”

10. Farming News Module

Purpose: This module automatically collects and displays recent agriculture news.

Technologies: Requests, Beautiful Soup, RSS/XML parsing and SQLite

Source: Krishi Jagran RSS feed

Request RSS feed ↓ Receive XML content ↓ Beautiful Soup parses it ↓ Extract article fields ↓ Remove duplicates ↓ Save in SQLite ↓ Display news cards

```
response = requests.get(url, headers=headers, timeout=10)
soup = BeautifulSoup(response.content, 'html.parser')
items = soup.find_all('item')

FarmingNews.objects.create(...)
```

Correct technical wording
It is accurate to say 'RSS feed parsing using Beautiful Soup'. It is similar to web scraping, but RSS parsing is more precise for this implementation.

“I used Python Requests and Beautiful Soup to read an agriculture RSS feed, extract news details and store them in SQLite.”

11. Profile Settings Module

Purpose: This module allows the farmer to update personal and farming details.

- First name and last name
- Email
- Phone number
- Village and district
- State
- Primary crop
- Land size

Technologies: Django Forms, Django Authentication and SQLite

Open profile ↓ Load existing information ↓ User edits form ↓ Django validates input ↓ User and profile records are updated

```
user_form = UserForm(request.POST, instance=request.user)
profile_form = UserProfileForm(request.POST, instance=profile)

user_form.save()
profile_form.save()
```

“The Profile module uses Django ModelForms to retrieve, validate and update the logged-in farmer's personal and farming details.”

12. AI Leaf Scanner - Quick Summary

This module was explained separately, but it is included here for completeness.

Upload or capture image ↓ Convert to RGB ↓ Resize to 224 x 224 ↓ Convert to numerical array ↓ MobileNetV2 predicts ↓ Highest class is selected ↓ Confidence percentage is shown

Main technologies: TensorFlow, Keras, MobileNetV2, Pillow, NumPy and H5.

```
predictions = model.predict(img_array)
pred_idx = np.argmax(predictions[0])
confidence = float(predictions[0][pred_idx]) * 100
```

"H5 is a file format used to save a trained deep-learning model, including what the model has learned."

13. Complete Technology List and Where It Is Used

Technology	Used for
Django	Main backend framework, URLs, views, forms, authentication and module connection
HTML, CSS, Bootstrap	Responsive frontend, cards, forms, navigation and page design
JavaScript	Interactive browser features, including webcam/capture support
SQLite	Stores users, profiles, crops, predictions, prices, weather history, news and recommendations
Django ORM	Create, read, update, delete, search and filter database records
Scikit-learn	Decision Tree, Random Forest and KNN disease prediction
TensorFlow and Keras	Deep-learning leaf image classification
MobileNetV2	Transfer-learning image-classification architecture
Pandas	Data cleaning, totals, averages, grouping, sorting and analytics
Plotly	Interactive pie charts, bar charts and line charts
OpenWeatherMap API	Current weather and forecast data
Government Data API	APMC mandi market-price records
JSON	Processing API responses and class-name data
Requests / urllib	Sending HTTP requests to external services
Beautiful Soup	Parsing farming-news RSS/XML content
Pillow	Opening, converting and resizing leaf images
NumPy	Converting images and model inputs into numerical arrays
ReportLab	Generating downloadable PDF reports
H5 / h5py	Saving and loading the trained deep-learning model

Best technology summary
Django is the main framework; SQLite stores project data; Scikit-learn performs environmental disease prediction; TensorFlow and MobileNetV2 classify leaf images; Pandas analyses data; Plotly draws charts; REST APIs provide weather and mandi prices; Beautiful Soup parses agriculture news; ReportLab creates PDFs.

14. Final Presentation Script

You can speak the following paragraph during your project presentation:

“I developed Smart Farmer Assistant using Django as the main web framework and SQLite as the database. I used Django authentication, forms, ORM and CRUD operations for user profiles and crop management. The dashboard summarizes the farmer's land, investment, revenue, crops and recent predictions. For crop analytics, I used Pandas to calculate totals and averages, and Plotly to create interactive pie, bar and line charts. I integrated the OpenWeatherMap REST API for weather and the Government of India Data API for APMC mandi prices; both return data that is processed by Python. I used Scikit-learn with Decision Tree, Random Forest and KNN for disease prediction from environmental inputs. For image-based leaf classification, I used TensorFlow, Keras and MobileNetV2. The fertilizer module uses rule-based recommendations, and the farming-news module uses Requests and Beautiful Soup to parse an agriculture RSS feed. I also used ReportLab to generate PDF reports.”

“My project combines Django web development, SQLite database operations, machine learning, deep learning, data analysis, interactive visualization, REST API integration, RSS parsing and PDF generation in one farmer-support system.”

15. Quick Faculty Questions and Answers

Q: Which framework did you use?
A: Django, because it provides URLs, views, forms, authentication, ORM and database integration.

Q: Which database did you use?
A: SQLite, which is Django's lightweight default relational database.

Q: Where is machine learning used?
A: In disease prediction from crop, soil, temperature, humidity and rainfall inputs.

Q: Which ML algorithms are used?
A: Decision Tree, Random Forest and K-Nearest Neighbours.

Q: Where is deep learning used?
A: In the AI Leaf Scanner for image classification using MobileNetV2.

Q: What is H5?
A: H5 is a file format used to save a trained deep-learning model, including what the model has learned.

Q: Where is Pandas used?
A: In crop analytics and market-data preparation for calculations and sorting.

Q: Where is Plotly used?
A: For interactive pie charts, bar charts and line charts.

Q: Do you use REST APIs?
A: Yes. OpenWeatherMap supplies weather data and the Government Data API supplies mandi prices.

Q: In which format do APIs return data?
A: JSON format.

Q: Where is Beautiful Soup used?
A: To parse the Krishi Jagran farming-news RSS feed.

Q: Is fertilizer guidance machine learning?
A: No. It is rule-based logic using predefined recommendations and soil multipliers.

Q: Is Crop Library machine learning?
A: No. It is a database-based information, search and filtering module.

Q: How do you protect user data?
A: Pages use login protection, and crop queries filter by the current logged-in user.

Q: What performs the final ML prediction?
A: The model.predict(features) line.